Indian Institute of Technology Bhubaneswar

School of Electrical Sciences

Autumn End-Semester Examination-2019

Subject Name: Compiler Design	Subject Code: CS4L001
Date: 20-11-2019	Full Marks: 85
Duration: 3 Hours	

Answer a new section on a fresh page (strictly) and all parts of a section together.

Section A: Answer the following questions briefly. Each question carries 3 marks. ----- 10*3 =30 Marks

A1. Discuss very briefly why the lexical analysis phase of a compiler is implemented separately. Briefly discuss regarding longest match rule in lexical analysis (via a simple example).

A2. Explain with an example the usage of syntehsized and inherited attributes. Discuss the role of the syntax-directed translation scheme.

A3. Explain what are the benefits of intermediate code generation? What are the various types of intermediate code representations?

A4. What is code optimization? Why is it needed? Mention and briefly discuss about a couple of optimization techniques.

A5. Describe the basic structure of an Activation Record (AR). Consider the following C program. Discuss if the activation records (ARs) for each of the functions can be allocated statically. Explain why or why not.

```
#include <stdio.h>
int counter = 0;
int calc (int a, int b) {
  int c;
  counter++;
  if (b==3) return (a*a*a);
  else {
    c = calc(a, b/3);
    return (c*c*c);
  }
}
int main (){
   calc(4, 81);
   printf ("%d", counter);
}
```

A6. Left factor the following grammar. Does left factoring make it suitable for top-down parsing?

S -> S S + | S S * | a

Scanned by CamScanner

A7. Represent the following 3-address code as: quadruples, abstract-syntax tree and directed-acyclic graph.

a + b*c - d/b*c

1. t1= b*c
2. t2=a+t1
3. t3=b*c
4. t4=d/t3
5. t5=t2-t3

A8. Consider the grammar with the following translation rules and **E** as the start symbol.

E → E1 # T	{ E.value = E1.value * T.value }
E → T	{ E.value = T.value }
T → T1 & F	{ T.value = T1.value + F.value }
T → F	{ T.value = F.value }
F → num	{ F.value = num.value }

Draw the annotated parse-tree (showing value of attributes) for the expression: **2 # 4 & 4 # 5 & 5**. What is **E.Value** for the root of the parse tree?

A9. Consider the following code snippet. What will be the value printed when:

```
int v;
void P1( )
    {v = v + 1;}
void P2(int x)
    {x = x + 5; P1( );}
main()
    {v = 4; P2(v); print(v);}
```

1. call-by-value mechanism is considered. 5
2. call-by-reference mechanism is considered. 10
3. call-by-value-result mechanism is considered. 9

9 5 2 ++

A10. Consider the grammar with the following translation rules and **E** as the start symbol.

R → + T **{print ('+');}** R | ε

T → num **{print (num.val);}**

Here **num** is a token that represents an integer and **num.val** represents the corresponding integer value. For input **9 + 5 + 2**, what will this translation scheme print?

3+6+3= 12 Marks

Section B: Top-down Parsing —

Consider the following grammar with non-terminals **N = {P, Q}**, **T = {b, e, s, ;}**, start symbol **P**, with the following set of productions:

P → b Q e

Q → Q ; s | s

B1. Eliminate left-recursion in this grammar by deriving an equivalent grammar that is non-left recursive. If the resulting grammar needs to be left-factored, derive an equivalent grammar that is also left-factored and thus has the **LL(1)** property.

B2. Compute *FIRST* and *FOLLOW* sets for each non-terminal. Construct **LL(1)** parse table.

B3. Show the actions of the **LL(1)** parser given the input string: **b s ; s ; s e**

Scanned by CamScanner

Section C: Bottom-up Parsing (LR(1), LALR(1)) — 4+3+3=10 Marks

Consider the following grammar with the set of non-terminals {**S, A, B**}, start symbol **S**, and terminal symbols {**a, b, c, d**}:

(1) S → Aa
(2) S → bAc
(3) S → Bc
(4) S → bBa
(5) A → d
(6) B → d

1, 5, 6, 9, 90, 11, 12

NOTE: Do not forget to augment the grammar with the production S' → S$

C1. Construct **LR(1)** automaton.
C2. Based on the **LR(1)** automaton, construct the **LR(1)** parsing table.
C3. Is this grammar **LALR(1)**? Justify.

Section D: Semantic Analysis, Syntax Directed Definition/Syntax Directed Translation– 8+10= 18 Marks

D1. Consider the following *syntax-directed definition* over the grammar defined by **G = ({S, A, Sign}, S, {',', '-','+', 'n'}, P)** with **P** the set of production and the corresponding semantic rules depicted below. There is a special terminal symbol "**n**" that is lexically matched by any string of one numeric digit and whose value is the numeric value of its decimal representation. For the non-terminal symbols in **G** we have defined two attributes, ***sign*** and ***value***. The non-terminal **A** has these two attributes whereas **S** only has the ***value*** attribute and **Sign** only has the ***sign*** attribute.

S → A Sign	{S.val = A.val; A.sign = Sign.sign; print(A.val);}
Sign → +	{Sign.sign = 1}
Sign → -	{Sign.sign = 0}
A → n	{A.val = value(n)}

A → A1 , n	{A1.sign = A.sign;
	if (A.sign = 1) then
	A.val = min(A1.val,value(n));
	else
	A.val = max(A1.val,value(n)); }

(1) Explain the overall operation of this syntax-directed definition and indicate (with a brief explanation) which of the attributes are either synthesized or inherited.

(2) Give an attributed parse tree for the source string "**5,2,3-**" and evaluate the attributes in the attributed parse tree depicting the order in which the attributes need to be evaluated (if more than one order is possible indicate one.)

D2. Consider the following grammar **G** that generates all strings of a's, b's and c's. Add attributes and semantic actions to this grammar and construct a **SDD/ SDT scheme** that takes strings of a's, b's and c's as input and produces as output the number of substrings in the input string that correspond to the pattern **a(a|b)*c+(a|b)*b**. For example, the translation of the input string "**abbcabcababc**" should be "**3**".

S → S a
S → S b
S → S c
S → a
S → b
S → c

Clearly mention/explain the semantic attributes for the grammar symbols, for each production of the grammar add a set of rules for evaluation of the semantic attributes. Explain/illustrate your solution via an example.

Scanned by CamScanner

Section E: Intermediate Representations (IR), IR Optimizations, Runtime Environments—3*5 =15 Marks

E1. Consider the three-address code below for a procedure with input/output arguments **p0** and **p1** and using several temporary variables. Determine the following:

- **Basic blocks** (clearly indicating the lines of code belonging to each block),
- **Control-flow graph** with each basic block considered as a node

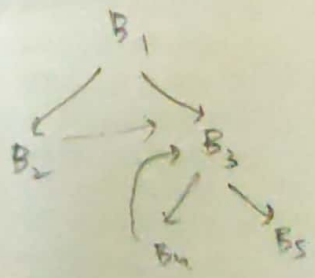
```
01:   t5 = 0
02:   t0 = p1
03:   t2 = 0
04:   t1 = p2
05:   if (t1 > 0) goto L1
06:   t5 = t0
07:   t0 = 1
08:   t2 = 0
09: L1: t1 = t0
10:   t4 = t0
11:   if (t4 > t5) goto L2
12:   t4 = t1 + 1
13:   t2 = t2 + 1
14:   goto L1
15: L2: t0 = p1
16:   t3 = t0
17:   t1 = t2 + t3
```

E2. Optimize the code below by apply the code transformations discussed (constant propagation, constant folding,.. etc). Indicate if any part of code is un-reachable. Give the resulting simplified code, and mention which variables can be eliminated and why.

```
      t1 = t1 + 1
L0:   t2 = 0
      t3 = t1 * 8 + 1
      t4 = t3 + t2
      t5 = t4 * 4
      t6 = *t5
      t7 = FP + t3
      *t7 = t2
      t8 = t1
      if (t8 > 0) goto L1
L1:   goto L0
L2:   t1 = 1
      t10 = 16
      t11 = t1 * 2
      goto L1
```

E3. In class we covered 3-address code generation for some high-level programming constructs. Based on the concepts discussed in class, provide 3-address code corresponding to the following code snippet (**NOTE**: it is not necessary to define grammar and SDT scheme etc. You can directly give the 3-address code corresponding to the given code snippet):

```
a = 3;
b = 4;
i = 0;
while (i < n)
{
  a = b + 1;
  a = a * a;
  i++;
}
c = a;
```



Scanned by CamScanner